

File Integrity Checker

Course: Computer Security **Project #18:** A tool that uses cryptographic hash functions to verify the integrity of files.

1. Project Overview

This project implements a **File Integrity Checker** in Python. The tool uses cryptographic hash functions (SHA-256 by default) to detect any unauthorized or accidental change in a set of files.

The idea is simple but powerful:

- Each file is passed through a hash function, which produces a fixed-length “digital fingerprint” of its contents.
- Any change — even one bit — produces a completely different hash (the *avalanche effect*).
- By saving a *baseline* of these fingerprints today and comparing them to fresh hashes later, we can prove whether files have been **modified**, **deleted**, or **added**.

This is the same idea behind real-world tools like **Tripwire**, **AIDE**, and the integrity checks used in package managers (apt, yum, npm).

2. Why This Matters in Computer Security

A file integrity checker provides three security guarantees:

Property	What it protects against
Integrity	Malware modifying system files, ransomware encryption
Detection	Unauthorized changes by intruders or insider threats
Auditability	Knowing exactly <i>what</i> changed and <i>when</i>

It does **not** provide confidentiality (the file content is still readable) and it does **not** provide authentication on its own — for that we would combine the hash with a digital signature (HMAC or RSA-signed hash).

3. How It Works

Step 1 — Hashing a single file

The program opens the file in binary mode, reads it in 4 KB chunks (so it works on any file size), and feeds each chunk into `hashlib`. The final `.hexdigest()`

is the file's fingerprint.

Step 2 — Creating a baseline

`scan_directory()` walks the folder recursively with `os.walk()` and records `{relative_path: hash}` for every file. The dictionary is saved to `baseline.json` along with metadata (directory, algorithm, timestamp).

Step 3 — Verifying integrity

The program re-scans the directory and compares the new hashes against the baseline:

- **Unchanged** : path exists in both, hashes match
- **Modified** : path exists in both, hashes differ
- **Deleted** : path was in baseline but no longer exists
- **New** : path exists now but not in the baseline

A clear report is printed showing each category.

4. Why SHA-256?

The project supports SHA-256, SHA-512, and MD5, but **SHA-256 is the default** because:

- It is **collision-resistant** — it is computationally infeasible to find two different files with the same hash.
 - It produces a 256-bit output, large enough to make brute-force collision attacks impractical.
 - It is part of the SHA-2 family standardized by NIST and is widely used (TLS certificates, Bitcoin, Linux package signing).
 - **MD5 and SHA-1 are broken** for collision resistance and should not be used for security purposes — the tool supports MD5 only for educational comparison.
-

5. How to Run

`python3 file_integrity_checker.py`

You'll see a menu:

1. Create baseline (snapshot a directory)
2. Verify integrity (compare against baseline)
3. Hash a single file
4. Exit

Typical demo flow

1. Choose **1** and point it at a folder → creates **baseline.json**.
 2. Manually edit a file, delete one, and add a new one.
 3. Choose **2** → the report will show exactly which files changed.
-

6. Sample Output

```
=====
                        INTEGRITY CHECK REPORT
=====
Baseline created : 2026-05-03T15:40:19
Algorithm       : SHA256
Files in baseline: 3
Files now       : 3
-----
  Unchanged : 1
  Modified  : 1
  Deleted   : 1
  New       : 1
=====

[!] MODIFIED FILES:
    - file1.txt
      old: 1894a19c85ba153acbf743ac4e43fc00...
      new: cac5c0b32d8262b0850569ad07e67df1...

[!] DELETED FILES:
    - file2.txt

[+] NEW FILES:
    - file4_new.txt
```

7. Limitations and Possible Improvements

- The baseline file itself is not protected — an attacker who tampers with files could also rewrite **baseline.json**. A real-world fix is to **sign** the baseline with a private key, or store it on read-only / offline media.
 - The tool does not watch files in real time. A scheduled task (cron) can run verification periodically.
 - Symbolic links and very large directory trees could be handled more efficiently with multi-threading.
-

8. Concepts Demonstrated

- Cryptographic hash functions (SHA-256)
- Avalanche effect and collision resistance
- File system traversal in Python
- Baseline/snapshot security model
- JSON serialization
- The integrity property of the CIA triad